

# ANJUMAN INSTITUTE OF TECHNOLOGY AND MANAGEMENT

AFFILIATED TO VISVESVARAYA TECHNOLOGICAL  
UNIVERSITY, BELAGAVI



**AITM , BHATKAL**

**Anjuman Abad, Belalkanda, Bhatkal,  
Karnataka - 581320**

**C PROGRAMMING LAB MANUAL – 1BPOPL107/207**

Prepared By :

**Mohammed Muaaz**

## PART – A

1. A robot needs to find how far it must travel between two points on a 2D plane. Develop a C program to calculate the straight-line distance between the given coordinates.

### Algorithm :

Step 1: Start

Step 2: Input the coordinates of the first point (x1, y1)

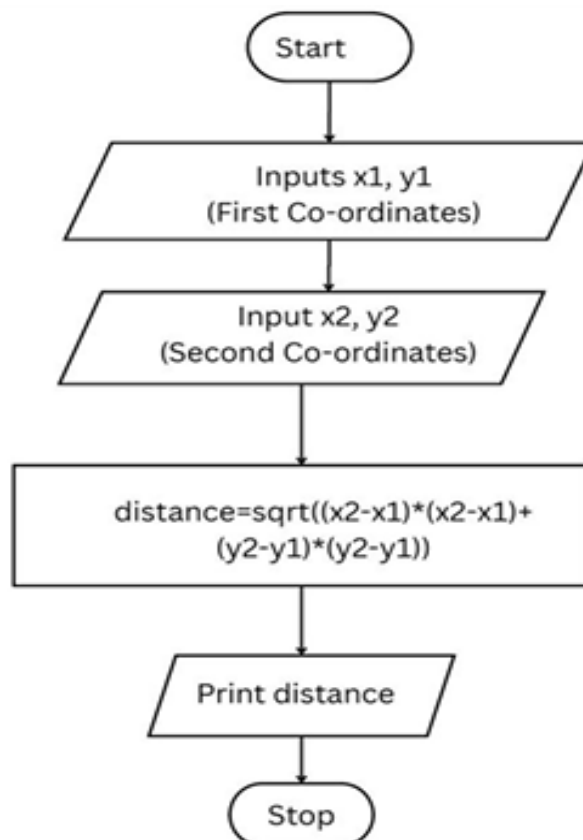
Step 3: Input the coordinates of the second point (x2, y2)

Step 4: Compute the distance  $\text{distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

Step 8: Display the distance

Step 9: Stop

### Flowchart :



### Program code :

```
#include <stdio.h>

#include <math.h>

int main()

{

    float x1, y1, x2, y2, distance;

    printf("Enter x1 and y1 (coordinates of the first point): ");

    scanf("%f %f", &x1, &y1);

    printf("Enter x2 and y2 (coordinates of the second point): ");

    scanf("%f %f", &x2, &y2);

    distance = sqrt((x2 - x1)*(x2 - x1) + (y2 - y1)*(y2 - y1));

    printf("The straight-line distance between the two points is: %.2f\n", distance);

}
```

### Output :

```
Enter x1 and y1 (coordinates of the first point):
0
0
Enter x2 and y2 (coordinates of the second point):
3
4
The straight-line distance between the two points is: 5.00
```

```
Enter x1 and y1 (coordinates of the first point):
1
1
Enter x2 and y2 (coordinates of the second point):
4
5
The straight-line distance between the two points is: 5.00
```

```
Enter x1 and y1 (coordinates of the first point):
-2
-3
Enter x2 and y2 (coordinates of the second point):
-4
-7
The straight-line distance between the two points is: 4.47
```

2. Develop a C program that takes a student's marks as input and displays their grade based on the following criteria: 90 and above: Grade A 75 to 89: Grade B 60 to 74: Grade C 50 to 59: Grade D Below 50: Grade F Choose a suitable control structure to implement this logic efficiently.

### Algorithm :

Step 1: Start

Step 2: Input the student's marks

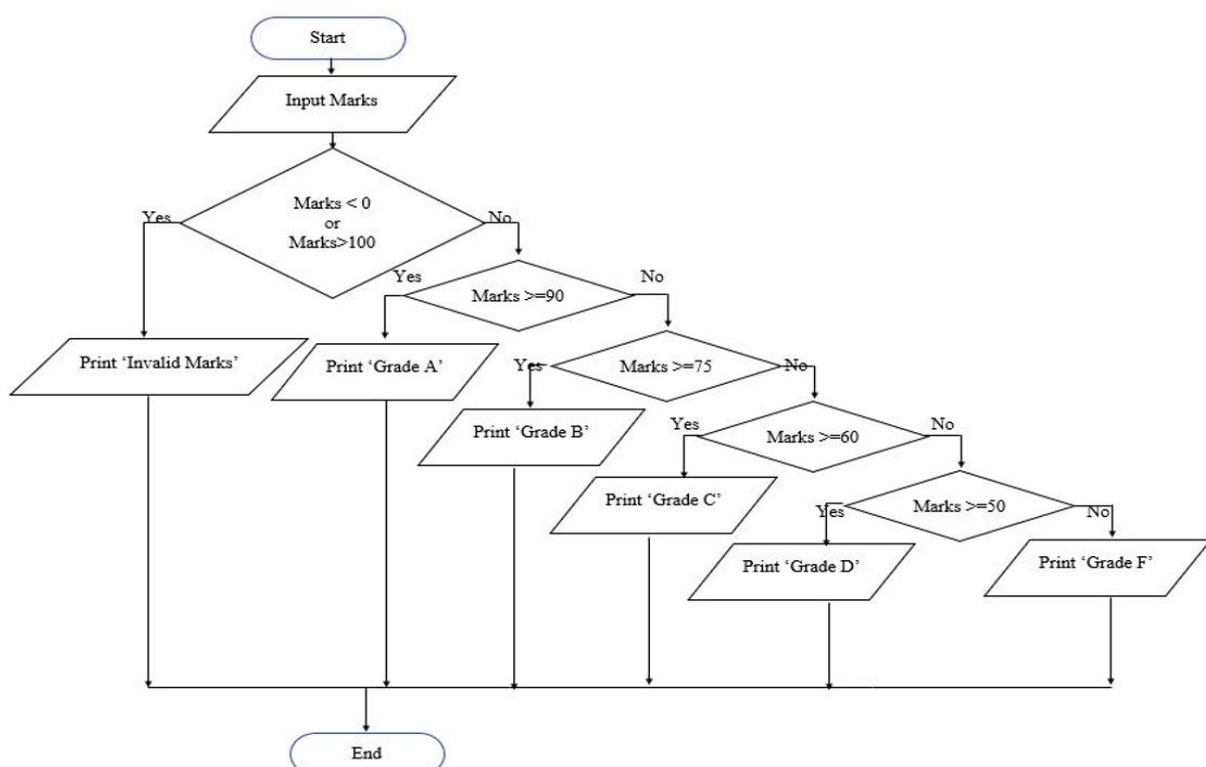
Step 3: Use if-else if statements to determine the grade:

- If marks  $\geq 90 \rightarrow$  Grade A
- Else if marks  $\geq 75 \rightarrow$  Grade B
- Else if marks  $\geq 60 \rightarrow$  Grade C
- Else if marks  $\geq 50 \rightarrow$  Grade D
- Else  $\rightarrow$  Grade F

Step 4: Display the grade

Step 5: Stop

### Flowchart :



### **Program Code :**

```
#include <stdio.h>

int main()
{
    int marks;
    printf("Enter the student's marks (0 to 100): ");
    scanf("%d", &marks);
    if (marks < 0 || marks > 100) {
        printf("Invalid marks! Please enter a value between 0 and 100.\n");
    }
    else if (marks >= 90) {
        printf("Grade: A\n");
    }
    else if (marks >= 75) {
        printf("Grade: B\n");
    }
    else if (marks >= 60) {
        printf("Grade: C\n");
    }
    else if (marks >= 50) {
        printf("Grade: D\n");
    }
    else {
        printf("Grade: F\n");
    }
    return 0;
}
```

## Output :

```
Enter the student's marks (0 to 100): 95  
Grade: A
```

```
Enter the student's marks (0 to 100): 80  
Grade: B
```

```
Enter the student's marks (0 to 100): 65  
Grade: C
```

```
Enter the student's marks (0 to 100): 55  
Grade: D
```

```
Enter the student's marks (0 to 100): 45  
Grade: F
```

```
Enter the student's marks (0 to 100): -20  
Invalid marks! Please enter a value between 0 and 100.
```

**3.** Develop a C program that takes a unique identification input like PAN Number, AADHAR\_Number, APAAR\_Id, Driving License, Passport and checks it against a set of stored KYC records. Based on the input, display whether the individual is verified or not. Use an appropriate control structure to handle multiple possible ID matches. Assume all Unique identification are of integer type.

**Algorithm :**

Step 1: Start

Step 2: Display the menu:

- 1 → PAN Number
- 2 → AADHAR Number
- 3 → APAAR Id
- 4 → Driving License
- 5 → Passport

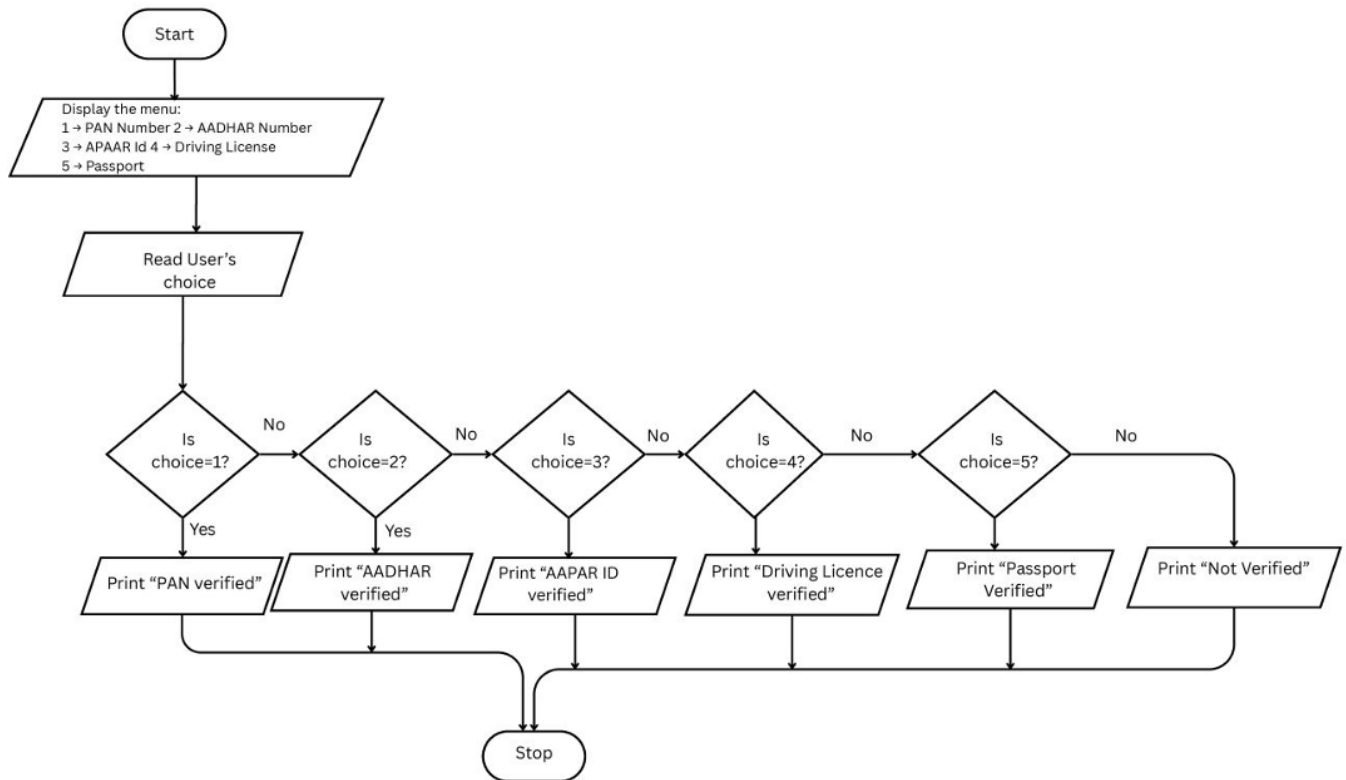
Step 3: Read the user's choice.

Step 4: Use a switch(choice) to process the selected ID type:

- Case 1 (PAN): Print "PAN Verified".
- Case 2 (AADHAR): Print "AADHAR Verified".
- Case 3 (APAAR): Print "APAAR Verified".
- Case 4 (Driving License): Print "Driving License Verified".
- Case 5 (Passport): Print "Passport Verified".
- Default: Print "Not verified".

Step 5: Stop

**Flowchart :**



### Program Code :

```

#include <stdio.h>

int main()
{
    int choice;
    printf("----- KYC Verification System -----\\n");
    printf("1. PAN Number\\n");
    printf("2. AADHAR Number\\n");
    printf("3. APAAR Id\\n");
    printf("4. Driving License\\n");
    printf("5. Passport\\n");
    printf("Enter your choice (1-5): ");
    scanf("%d", &choice);
    switch(choice)
    {
        case 1: // PAN
  
```

```
        printf("PAN Verified!\n");
        break;
case 2: // AADHAAR
        printf("AADHAR Verified!\n");
        break;
case 3: // APAAR
        printf("APAAR Verified!\n");
        break;
case 4: // Driving License
        printf("Driving License Verified!\n");
        break;
case 5: // Passport
        printf("Passport Verified!\n");
        break;
default:
        printf("Not Verified!\n");
}

return 0;
}
```

### Output :

```
----- KYC Verification System -----
1. PAN Number
2. AADHAR Number
3. APAAR Id
4. Driving License
5. Passport
Enter your choice (1-5): 2
AADHAR Verified!
```

```
Enter your choice (1-5): 5
Passport Verified!
```

```
Enter your choice (1-5): 7
Not Verified!
```

4. A math app needs to determine the type of roots for a quadratic equation based on user input. Develop a C program to calculate and display the roots based on the given coefficients.

**Algorithm :**

Step1: Start

Step2: Input the coefficients a, b, and c of the quadratic equation.

Step3: Check if  $a == 0$  and  $b == 0$ :

    If true, print "Invalid Coefficients" and terminate the program.

Step4: Else if  $a == 0$ :

    The equation is linear ( $bx + c = 0$ ).

    Calculate  $\text{root} = -c / b$ .

    Print the linear root.

Step5: Else ( $a \neq 0$ ):

    The equation is quadratic.

    Calculate the discriminant:  $\text{disc} = b^2 - 4ac$ .

    Print the discriminant.

    If  $\text{disc} > 0$ :

        Calculate two real and distinct roots:

$\text{root1} = (-b + \sqrt{\text{disc}}) / (2a)$

$\text{root2} = (-b - \sqrt{\text{disc}}) / (2a)$

            Print both roots.

    Else if  $\text{disc} == 0$ :

        Calculate one real and repeated root:

$\text{root} = -b / (2a)$

            Print the root.

    Else ( $\text{disc} < 0$ ):

        Calculate complex roots:

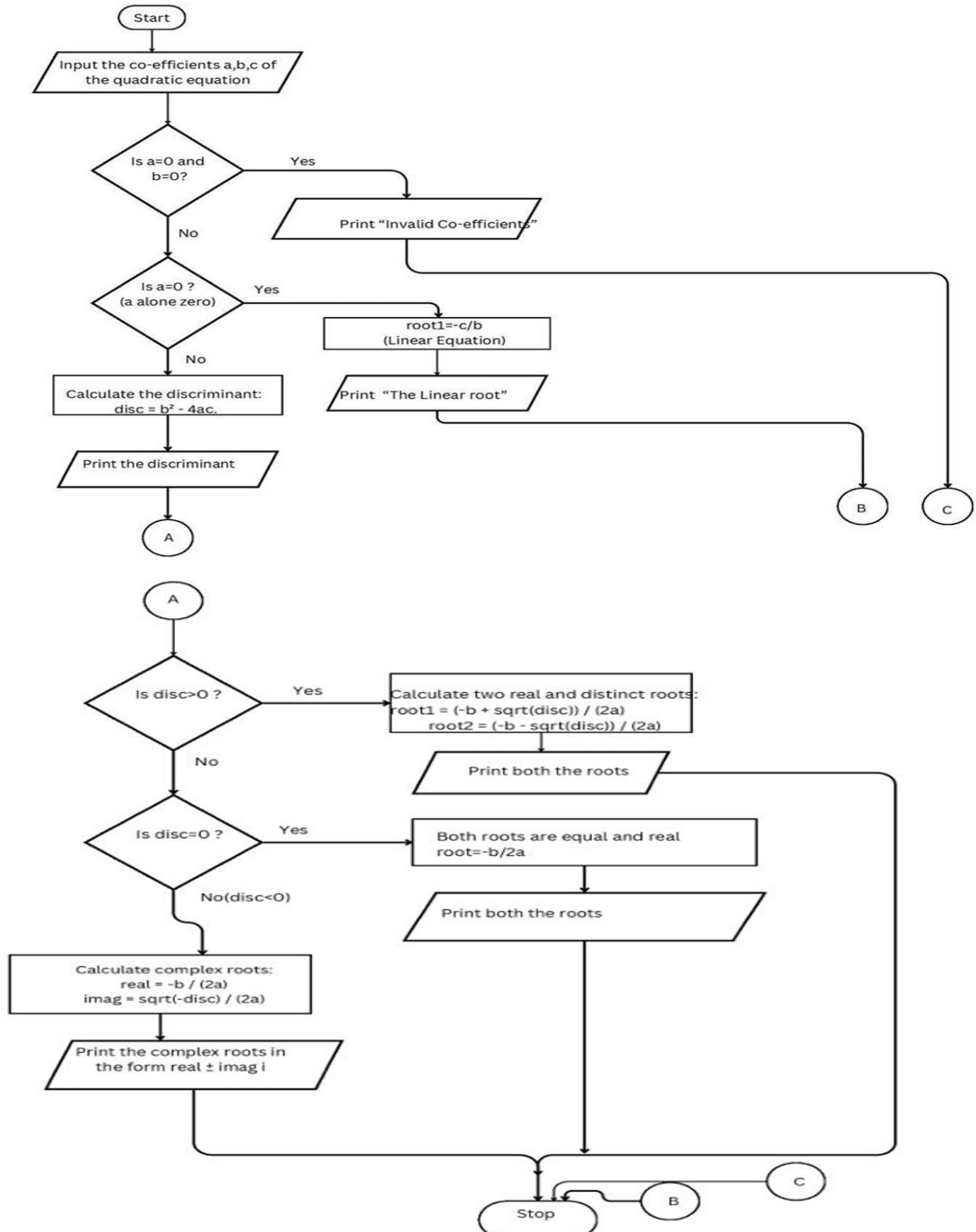
$\text{real} = -b / (2a)$

$$\text{imag} = \sqrt{-\text{disc}} / (2a)$$

Print the complex roots in the form  $\text{real} \pm \text{imag } i$ .

Step6: Stop

### Flowchart :



### Program Code :

```
#include <stdio.h>

#include <math.h>

int main()

{
    float a, b, c, root1, root2, disc, real, imag;

    printf("Enter coefficients of quadratic equation (a, b, c): ");

    scanf("%f %f %f", &a, &b, &c);

    if (a == 0 && b == 0)
    {
        printf("Invalid Coefficients!\n");
    }
    else if (a == 0)
    {
        // Linear equation bx + c = 0
        root1 = -c / b;
        printf("Linear Equation with root: %.2f\n", root1);
    }
    else
    {
        // Quadratic equation
        disc = b * b - 4 * a * c;
        printf("Discriminant = %.2f\n", disc);
        if (disc > 0)
        {
            root1 = (-b + sqrt(disc)) / (2 * a);
            root2 = (-b - sqrt(disc)) / (2 * a);
            printf("Two distinct real roots: %.2f and %.2f\n", root1, root2);
```

```

    }
    else if (disc == 0)
    {
        root1 = -b / (2 * a);
        printf("One real root: %.2f\n", root1);
    }
    else
    {
        real = -b / (2 * a);
        imag = sqrt(-disc) / (2 * a);
        printf("Complex roots: %.2f + %.2fi and %.2f - %.2fi\n", real, imag, real, imag);
    }
}

return 0;
}

```

### Output :

```

Enter coefficients of quadratic equation (a, b, c):
1
-3
2
Discriminant = 1.00
Two distinct real roots: 2.00 and 1.00

```

```

Enter coefficients of quadratic equation (a, b, c):
1 2 5
Discriminant = -16.00
Complex roots: -1.00 + 2.00i and -1.00 - 2.00i

```

```

Enter coefficients of quadratic equation (a, b, c):
0 2 1
Linear Equation with root: -0.50

```

```

Enter coefficients of quadratic equation (a, b, c):
0 0 1
Invalid Coefficients!

```

5. A sensor in a robotic arm needs to calculate the angle of rotation in real-time, but the hardware doesn't support built-in trigonometric functions. Develop a C program to approximate the value of  $\sin(x)$  using a series expansion method for improved performance.

### Algorithm :

Step 1: Start

Step 2: Input angle x in degrees

Step 3: Convert degrees to radians:  $x\_rad = x * \pi / 180$

Step 4: Initialize variables: - term = x\_rad - sum = 0 - sign = 1

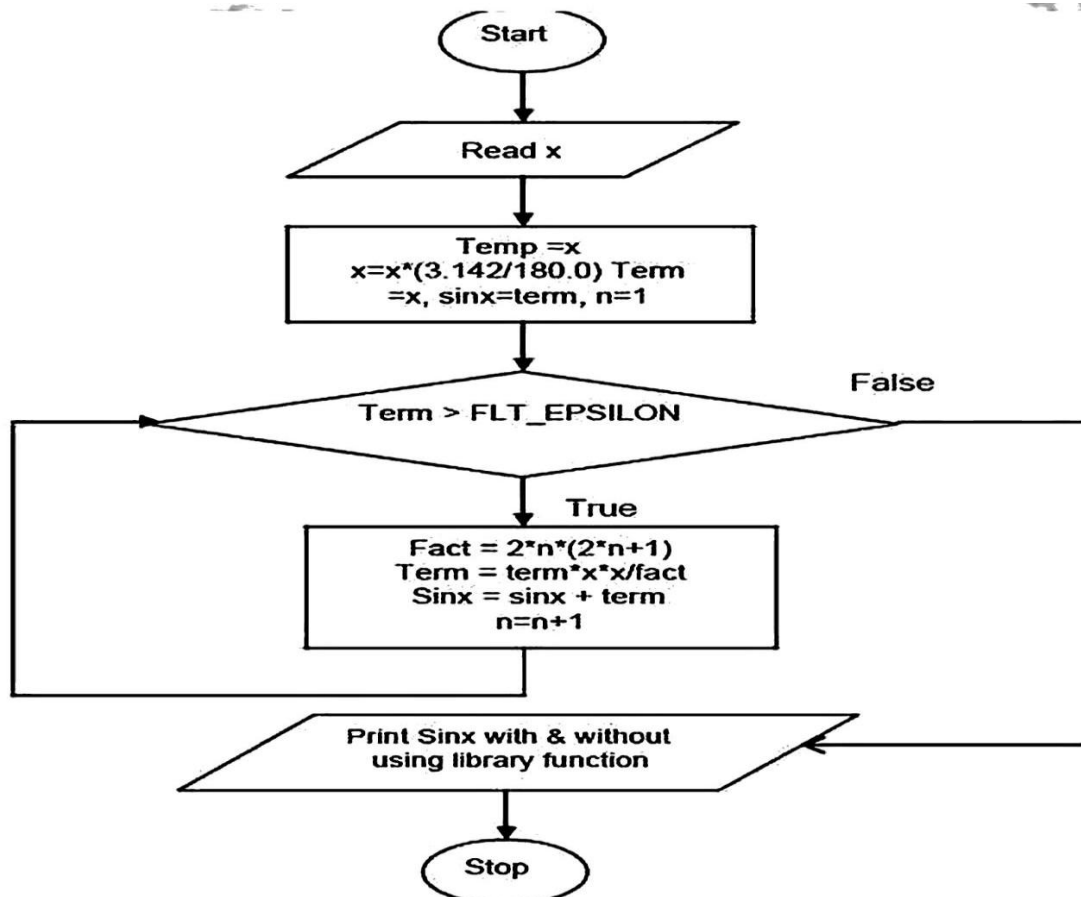
Step 5: Loop through n terms for the desired accuracy - Add current term \* sign to sum - Compute next term using formula:

$$\text{term} = \text{term} * x\_rad * x\_rad / ((2*i) * (2*i+1))$$

Step 6: Print final sum as approximate  $\sin(x)$

Step 7: Stop

### Flowchart :



### Program Code :

```
#include <stdio.h>

#include <math.h>

#define PI 3.14

int main()
{
    float sum,term,x,nume;
    int deg, i=2, fact=1;
    printf("Enter angle in degrees: \n");
    scanf("%d", &deg);
    x=(deg*PI)/180;
    sum=x;
    nume=x;
    do
    {
        fact=fact*i*(i+1);
        nume = -nume * x * x ;
        term = nume/fact;
        sum += term;
        i+=2;
    } while (fabs(term)>=0.0001);
    printf("Approximated sin(%d) = %.2f\n", deg, sum);
}
```

### Output :

```
Enter angle in degrees:
30
Approximated sin(30) = 0.50
```

```
Enter angle in degrees:  
90  
Approximated sin(90) = 1.00
```

```
Enter angle in degrees:  
0  
Approximated sin(0) = 0.00
```

```
Enter angle in degrees:  
180  
|
```

== > (No output) ~0.0 (close to zero, slight floating error)  
 $\sin(180^\circ) \approx 0$

6. Develop a C program that accepts a course description string and a keyword from the user. Search whether the keyword exists within the course description using appropriate string functions. If found, display: "Keyword " found in the course description." Otherwise, display: "Keyword " not found in the course description.

### Algorithm :

Step 1: Start

Step 2: Input the course description (string)

Step 3: Input the keyword to search

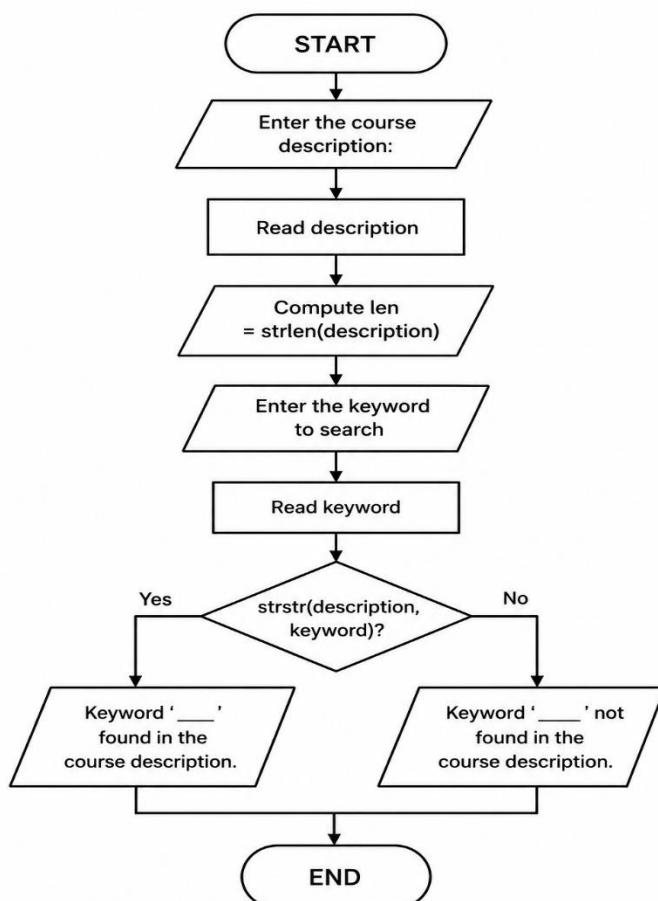
Step 4: Use the strstr() function to check if the keyword exists in the course

description - If strstr() returns a non-NULL pointer → Keyword found - Else  
→ Keyword not found

Step 5: Display appropriate message

Step 6: Stop

### Flowchart :



### Program Code :

```
#include <stdio.h>

#include <string.h>

#define MAX 100

int main()
{
    char description[MAX];
    char keyword[100];
    printf("Enter the course description:\n");
    fgets(description,MAX,stdin);
    printf("Enter the keyword to search: ");
    scanf("%s", keyword);
    if (strstr(description, keyword))
    {
        printf("Keyword '%s' found in the course description.\n",keyword);
    }
    else
    {
        printf("Keyword '%s' not found in the course description.\n",keyword);
    }
}
```

### Output :

```
Enter the course description:
Introduction to computer science and programming
Enter the keyword to search: programming
Keyword 'programming' found in the course description.
```

```
Enter the course description:
Fundamentals of network
Enter the keyword to search: aitm
Keyword 'aitm' not found in the course description.
```

7. Develop a C program that takes marks for three subjects as input. Use a function to check if the student has passed (minimum 40 marks in each subject). Display the average and whether the student passed or failed. Problem Description

### Algorithm :

Step 1: Start

Step 2: Input marks for three subjects: mark1, mark2, mark3

Step 3: Calculate the average:  $\text{average} = (\text{mark1} + \text{mark2} + \text{mark3}) / 3.0$

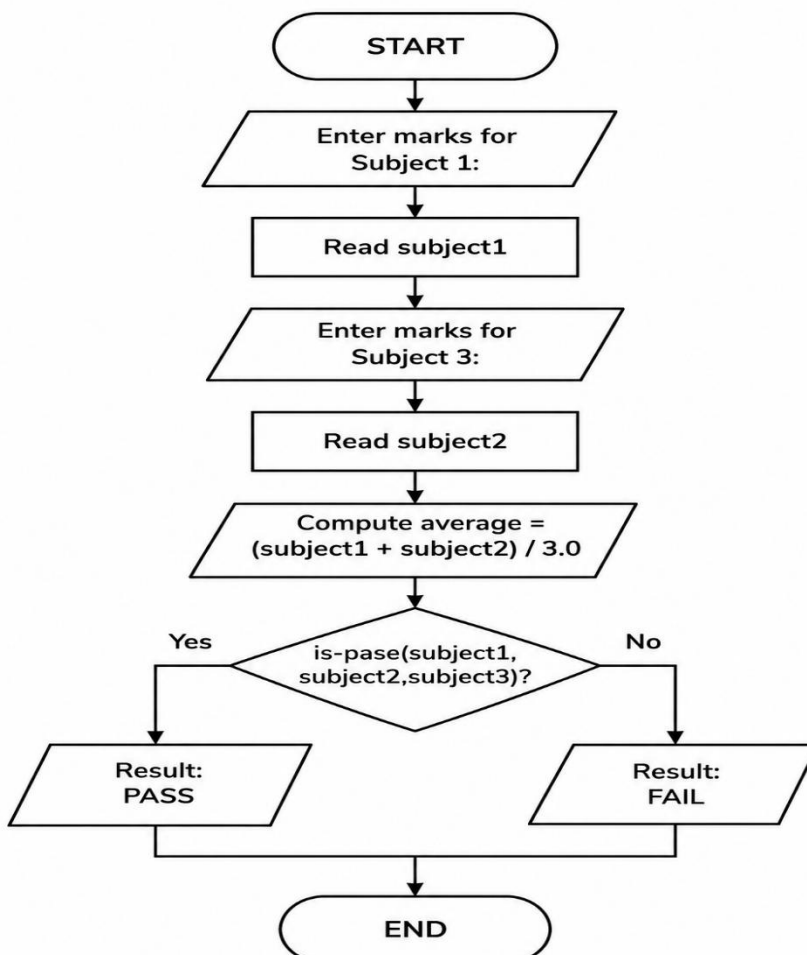
Step 4: Call a function isPassed(mark1, mark2, mark3) that returns:

- 1 if all marks  $\geq 40$
- 0 otherwise

Step 5: Display the average

Step 6: If return value is 1  $\rightarrow$  Print "Passed"

### Flowchart :



### **Program Code :**

```
#include <stdio.h>

int isPassed(int m1, int m2, int m3)
{
    if (m1 >= 40 && m2 >= 40 && m3 >= 40)
    {
        return 1;
    }
    else
    {
        return 0;
    }
}

int main()
{
    int marks1, marks2, marks3;
    float average;
    printf("Enter marks for Subject 1: ");
    scanf("%d", &marks1);
    printf("Enter marks for Subject 2: ");
    scanf("%d", &marks2);
    printf("Enter marks for Subject 3: ");
    scanf("%d", &marks3);
    average = (marks1 + marks2 + marks3) / 3.0;
    printf("Average Marks: %.2f\n", average);
    if (isPassed(marks1, marks2, marks3))
    {
        printf("Result: PASS\n");
    }
}
```

```
}  
  
else  
  
{  
    printf("Result: FAIL\n");  
}  
}
```

### Output :

```
Enter marks for Subject 1: 50  
Enter marks for Subject 2: 60  
Enter marks for Subject 3: 70  
Average Marks: 60.00  
Result: PASS
```

```
Enter marks for Subject 1: 30  
Enter marks for Subject 2: 40  
Enter marks for Subject 3: 50  
Average Marks: 40.00  
Result: FAIL
```

```
Enter marks for Subject 1: 80  
Enter marks for Subject 2: 90  
Enter marks for Subject 3: 50  
Average Marks: 73.33  
Result: PASS
```

**8.** In an ATM system, two account balances need to be swapped temporarily for validation. Develop a C program that accepts two balances and uses a function with pointers to swap them. Display the balances before and after swapping.

**Algorithm :**

Step 1: Start

Step 2: Input two account balances: balance1 and balance2

Step 3: Display balances before swapping

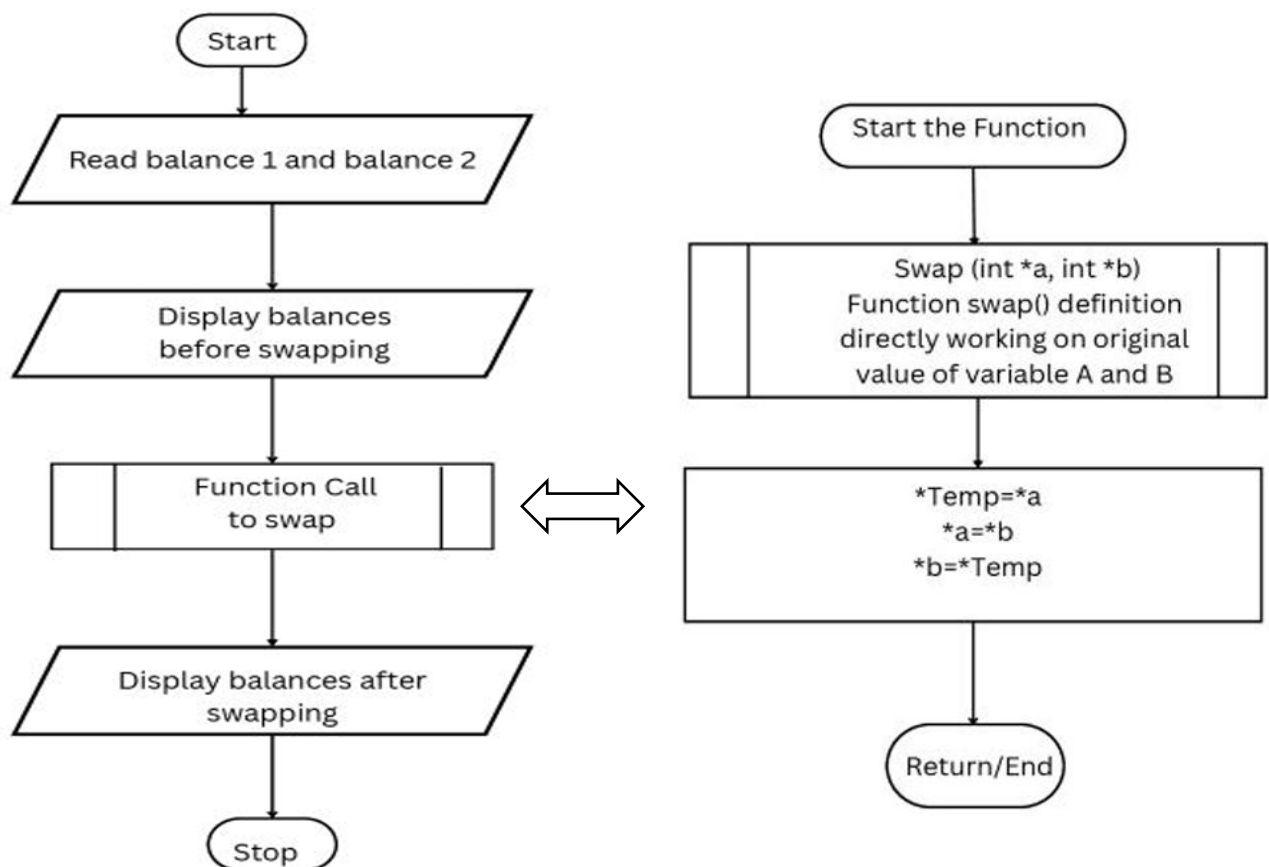
Step 4: Call a function swap(&balance1, &balance2) that:

- Uses a temporary variable and pointer dereferencing to swap the values

Step 5: Display balances after swapping

Step 6: Stop

**Flowchart :**



**Program Code :**

```
#include <stdio.h>

void swap(float *a, float *b)
{
    float temp = *a;
    *a = *b;
    *b = temp;
}

int main()
{
    float balance1, balance2;
    printf("Enter balance for Account 1: ");
    scanf("%f", &balance1);
    printf("Enter balance for Account 2: ");
    scanf("%f", &balance2);
    printf("\nBefore Swapping:\n");
    printf("Account 1 Balance: %.2f\n", balance1);
    printf("Account 2 Balance: %.2f\n", balance2);
    swap(&balance1, &balance2);
    printf("\nAfter Swapping:\n");
    printf("Account 1 Balance: %.2f\n", balance1);
    printf("Account 2 Balance: %.2f\n", balance2);
}
```

## Output :

```
Enter balance for Account 1: 1000.50
Enter balance for Account 2: 2000.75
```

```
Before Swapping:
Account 1 Balance: 1000.50
Account 2 Balance: 2000.75
```

```
After Swapping:
Account 1 Balance: 2000.75
Account 2 Balance: 1000.50
```

```
Enter balance for Account 1: -100
Enter balance for Account 2: 100.3
```

```
Before Swapping:
Account 1 Balance: -100.00
Account 2 Balance: 100.30
```

```
After Swapping:
Account 1 Balance: 100.30
Account 2 Balance: -100.00
```

## PART – B

### **Program 1:** Book Availability in Library (Binary Search)

**Problem Statement :** A college library has a digital bookshelf system where each book is assigned a unique Book ID. The bookshelf is organized in ascending order of Book IDs. The task is to develop a C program to quickly find whether a book with a specific Book ID is available in the shelf using binary search.

#### **Algorithm :**

Step 1. Start

Step 2. Input n.

Step 3. Input n Book IDs into array books[].

Step 4. Input key.

Step 5. Set low = 0, high = n-1, found = 0.

Step 6. Repeat while low <= high: o mid = (low + high) / 2.

    If books[mid] == key: set found = 1 and break.

    Else if books[mid] < key: low = mid + 1.

    Else: high = mid - 1.

Step 7. If found == 1: print "Book is available".

Step 8. Else: print "Book is not available".

Step 9. Stop

#### **Program Code:**

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, i, key, low, high, mid, found = 0;
```

```
    printf("Enter number of books: ");
```

```
    scanf("%d", &n);
```

```
    int books[n];
```

```
printf("Enter %d Book IDs in ascending order:\n", n);
for(i = 0; i < n; i++)
{
    scanf("%d", &books[i]);
}
printf("Enter Book ID to search: ");
scanf("%d", &key);
low = 0;
high = n - 1;
while(low <= high)
{
    mid = (low + high) / 2;
    if(books[mid] == key)
    {
        found = 1;
        break;
    }
    else if(books[mid] < key)
    {
        low = mid + 1;
    }
    else
    {
        high = mid - 1;
    }
}
if(found)
{
```

```
        printf("Book is available.\n");
    }
    else
    {
        printf("Book is not available.\n");
    }
    return 0;
}
```

### Output :

```
Enter number of books: 4
Enter 4 Book IDs in ascending order:
101
115
120
130
Enter Book ID to search: 120
Book is available.
```

```
Enter number of books: 4
Enter 4 Book IDs in ascending order:
101
110
120
130
Enter Book ID to search: 105
Book is not available.
```

## **Program 2:** Sorting Scores in Descending Order (Bubble Sort)

**Problem Statement :** A sports teacher has recorded the scores of students in a 100-meter race. To prepare the result sheet, the teacher wants the scores arranged in descending order (from highest to lowest). The task is to develop a C program to sort the scores using Bubble Sort.

### **Algorithm :**

Step 1. Start

Step 2. Input number of students n.

Step 3. Input n scores into array scores[].

Step 4. For i = 0 to n-2:

    For j = 0 to n-i-2:

        If scores[j] < scores[j+1], swap them.

Step 5. Print sorted scores in descending order.

Step 6. Stop

### **Program Code :**

```
#include <stdio.h>

int main()
{
    int n, i, j, temp;
    printf("Enter number of students: ");
    scanf("%d", &n);
    int scores[n];
    printf("Enter the scores:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &scores[i]);
    }
}
```

```

// Bubble Sort (Descending Order)
for(i = 0; i < n-1; i++)
{
    for(j = 0; j < n-i-1; j++)
    {
        if(scores[j] < scores[j+1])
        {
            temp = scores[j];
            scores[j] = scores[j+1];
            scores[j+1] = temp;
        }
    }
}

printf("Scores in descending order:\n");
for(i = 0; i < n; i++)
{
    printf("%d ", scores[i]);
}

return 0;
}

```

### Output :

```

Enter number of students: 4
Enter the scores:
20
36
45
62
Scores in descending order:
62 45 36 20

```

```

Enter number of students: 4
Enter the scores:
5
5
5
5
Scores in descending order:
5 5 5 5

```

### **Program 3:** Warehouse Revenue Calculation.

**Problem Statement :** A small warehouse tracks many units of different products shipped from multiple branches. Another dataset shows how much revenue each product generates per unit. The task is to develop a C program that combines these datasets to calculate the total revenue generated by each branch.

#### **Algorithm :**

Step 1. Start

Step 2. Input number of branches (branches).

Step 3. Input number of products (products).

Step 4. Declare units[branches][products] and revenue[products].

Step 5. Input units shipped for each branch and product.

Step 6. Input revenue per unit for each product.

Step 7. For each branch i:

    Initialize total = 0.

    For each product j:

        Compute total += units[i][j] \* revenue[j].

    Print total revenue for branch i.

Step 8. Stop

#### **Program Code :**

```
#include <stdio.h>
```

```
int main() {
```

```
    int branches, products, i, j;
```

```
    printf("Enter number of branches: ");
```

```
    scanf("%d", &branches);
```

```
    printf("Enter number of products: ");
```

```
    scanf("%d", &products);
```

```
    int units[branches][products];
```

```
int revenue[products];
printf("\nEnter number of units shipped by each branch for each product:\n");
for(i = 0; i < branches; i++)
{
    printf("Branch %d:\n", i+1);
    for(j = 0; j < products; j++)
    {
        printf(" Product %d units: ", j+1);
        scanf("%d", &units[i][j]);
    }
}
// Input revenue per product
printf("\nEnter revenue per unit for each product:\n");
for(j = 0; j < products; j++)
{
    printf("Product %d revenue: ", j+1);
    scanf("%d", &revenue[j]);
}
printf("Total revenue per branch:\n");
for (i = 0; i < branches; i++)
{
    int total = 0;
    for (j = 0; j < products; j++)
    {
        total += units[i][j] * revenue[j];
    }
    printf("Branch %d: %d\n", i + 1, total);
}
```

```
    return 0;  
}
```

### Output :

```
Enter number of branches: 3  
Enter number of products: 2  
  
Enter number of units shipped by each branch for each product:  
Branch 1:  
    Product 1 units: 20  
    Product 2 units: 10  
Branch 2:  
    Product 1 units: 20  
    Product 2 units: 30  
Branch 3:  
    Product 1 units: 50  
    Product 2 units: 20  
  
Enter revenue per unit for each product:  
Product 1 revenue: 20  
Product 2 revenue: 30  
Total revenue per branch:  
Branch 1: 700  
Branch 2: 1300  
Branch 3: 1600
```

#### **Program 4: Mobile Contact Manager (Concatenating Names).**

**Problem Statement :** A basic mobile contact manager stores first and last names separately. For displaying the full names in the contact list, the system needs to join them manually. Additionally, the system must check the length of each full name to ensure it fits within the screen. The task is to implement these operations in C without using built-in string functions.

#### **Algorithm :**

Step 1. Start

Step 2. Input first name into first[].

Step 3. Input last name into last[].

Step 4. Initialize variables: o  $i = 0$ ,  $j = 0$ ,  $length = 0$ .

Step 5. Copy characters of first[] into full[], incrementing length.

Step 6. Insert a space character into full[length]. Increment length.

Step 7. Copy characters of last[] into full[], incrementing length.

Step 8. Insert '\0' at the end of full[].

Step 9. Print the full name and its length.

Step 10. If  $length > 20 \rightarrow$  Print "Name too long for screen."

Step 11. Else  $\rightarrow$  Print "Name fits the screen."

Step 12. Stop

#### **Program Code :**

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char first[50], last[50], full[100];
```

```
    int i = 0, j = 0, length = 0;
```

```
    printf("Enter first name: ");
```

```
    scanf("%s", first);
```

```
    printf("Enter last name: ");
```

```
scanf("%s", last);  
// Copy first name  
while(first[i] != '\0')  
{  
    full[length++] = first[i++];  
}  
// Add space  
full[length++] = ' ';  
// Copy last name  
while(last[j] != '\0')  
{  
    full[length++] = last[j++];  
}  
// Terminate string  
full[length] = '\0';  
// print output  
printf("Full Name: %s\n", full);  
printf("Length: %d\n", length);  
if(length > 20)  
{  
    printf("Name too long for screen.\n");  
}  
else  
{  
    printf("Name fits the screen.\n");  
}  
return 0;  
}
```

## Output :

```
Enter first name: John  
Enter last name: Doe  
Full Name: John Doe  
Length: 8  
Name fits the screen.
```

```
Enter first name: supercalipragilistic  
Enter last name: martinthomas  
Full Name: supercalipragilistic martinthomas  
Length: 33  
Name too long for screen.
```

## **Program 5: Currency Exchange Simulation (Call by Value and Call by Reference)**

**Problem Statement :** A currency exchange booth allows users to convert between two currencies. Before confirming the exchange, the system simulates a swap of the values (preview) without actually changing the original data. In other cases, it updates the actual values (permanent swap). The task is to implement both behaviors using Call by Value and Call by Reference in C.

### **Algorithm :**

Step 1. Start

Step 2. Input values x and y.

Step 3. Call swapByValue(x, y) and display preview result.

Step 4. Print x and y after call by value.

Step 5. Call swapByReference(&x, &y) and display actual result.

Step 6. Print updated x and y.

Step 7. Stop

### **Program Code :**

```
#include <stdio.h>
```

```
void swapByValue(int a, int b)
```

```
{  
    int temp = a;  
    a = b;  
    b = temp;  
    printf("Preview Swap (By Value): %d %d\n", a, b);  
}
```

```
void swapByReference(int *a, int *b)
```

```
{  
    int temp = *a;  
    *a = *b;
```

```

    *b = temp;

    printf("Actual Swap (By Reference): %d %d\n", *a, *b);
}

int main()
{
    int x, y;

    printf("Enter two currency values: \n");
    scanf("%d %d", &x, &y);
    swapByValue(x, y);
    printf("After Call by Value: %d %d\n", x, y);
    swapByReference(&x, &y);
    printf("After Call by Reference: %d %d\n", x, y);
    return 0;
}

```

Output :

```

Enter two currency values:
10
20
Preview Swap (By Value): 20 10
After Call by Value: 10 20
Actual Swap (By Reference): 20 10
After Call by Reference: 20 10

```

```

Enter two currency values:
50
75
Preview Swap (By Value): 75 50
After Call by Value: 50 75
Actual Swap (By Reference): 75 50
After Call by Reference: 75 50

```

## **Program 6:** Library Book Details (Structures in C).

**Problem Statement :** A local library needs to store and display details of its books, including title, author, and year of publication. The task is to design a structure that can hold these details and develop a C program to display the list of books entered.

### **Algorithm :**

Step 1. Start

Step 2. Define structure Book with fields: title, author, year.

Step 3. Input number of books n.

Step 4. Declare array of structure books[n].

Step 5. For each book: input title, author, and year.

Step 6. For each book: display "Book i: title by author (year)".

Step 7. Stop

### **Program Code :**

```
#include <stdio.h>

struct Book
{
    char title[50];
    char author[50];
    int year;
};

int main()
{
    int n, i;
    printf("Enter number of books: ");
    scanf("%d", &n);
    struct Book booklist[n];
    for(i = 0; i < n; i++)
```

```

{
    printf("\nEnter title for book %d:\n", i+1);
    scanf("%s", booklist[i].title);
    printf("Enter author for book %d:\n", i+1);
    scanf("%s", booklist[i].author);
    printf("Enter year for book %d:\n", i+1);
    scanf("%d", & booklist[i].year);
}
printf("\nLibrary Book Details:\n");
for(i = 0; i < n; i++)
{
    printf("Book %d: %s by %s (%d)\n", i+1, booklist[i].title, booklist[i].author,
    booklist[i].year);
}
return 0;
}

```

### Output :

```

Enter number of books: 2
Enter title for book 1:
Programming
Enter author for book 1:
Dennis
Enter year for book 1:
2010

Enter title for book 2:
AI
Enter author for book 2:
Russel
Enter year for book 2:
2013

Library Book Details:
Book 1: Programming by Dennis (2010)
Book 2: AI by Russel (2013)

```

```

Enter number of books: 1
Enter title for book 1:
DBMS
Enter author for book 1:
Navathe
Enter year for book 1:
2015

Library Book Details:
Book 1: DBMS by Navathe (2015)

```